RIZZ RADIO: ESTACIÓN DIGITAL EN VIVO, PLAYLIST DINÁMICO Y REPRODUCTOR MODERNO EN VS CODE

Propósito: Desarrollar una estación de radio digital moderna llamada **"RIZZ RADIO"** mediante la creación incremental de bloques de programación por horas (canciones, spots, jingles y monólogos) ensamblados en 123apps. Optimizaremos el tiempo con plantillas reutilizables y un sistema de reproducción avanzada con controles tipo streaming y loop infinito en VS Code.

1. Estrategia Pedagógica: Parrilla de Programación Modular y Trabajo Eficiente

Para evitar que el alumno dedique muchísimas horas a la creación de contenido desde cero y optimizar al máximo su tiempo, la estación de radio se construirá de forma incremental y estratégica, enfocando todo el esfuerzo en **una sola estación de radio institucional** a través de las siguientes técnicas clave:

1. **Uso de Plantillas Reutilizables en 123apps:** Una vez montado el primer bloque de 1 hora en 123apps, el alumno no necesita editar desde cero. Guarda esa estructura como plantilla y simplemente sustituye elementos específicos (por ejemplo, cambia una canción o un spot) manteniendo los jingles, cortinillas y fondo ya alineados. Esto permite generar un nuevo track de 1 hora en cuestión de minutos.
2. **Uso de Recursos y Música Sin Derechos de Autor:** El alumno puede incorporar música generada libremente en Suno, así como audios de librerías gratuitas o plataformas de IA para no tener que generar absolutamente todo el material por su cuenta.
3. **Conversión Instantánea de Textos Libres a Audio:** Se pueden utilizar textos en español libres de derechos (cuentos clásicos en *Wikisource*, artículos educativos de *Wikimedia*, o relatos de *Cervantes Virtual*) para copiarlos, pasarlos por un convertidor de Texto a Voz (Clipchamp, Descript o TTS) y obtener un monólogo profesional en segundos.
4. **Ajuste de Tiempo Exacto con IA:** El alumno puede pedirle a Gemini/ChatGPT que redacte un guion o locución con la duración exacta que necesita reemplazar en su plantilla de 123apps.
5. **Uso de Fragmentos de Audio Externos:** Se pueden integrar fragmentos de audio de diversas fuentes respetando las normas de uso justo (*Fair Use*) o licencias Creative Commons.
6. **Colaboración e Intercambio de Audios:** Se promueve la colaboración pidiendo audios a compañeros, amigos o familiares para intercambiar spots, entrevistas y saludos, nutriendo la estación sin cargar de trabajo a un solo estudiante.

2. Cómo Funciona el Sistema de Loop y Playlist Dinámico

- **Bloques de 1 Hora:** El alumno edita su bloque de 60 minutos en 123apps usando su plantilla optimizada, lo exporta a MP3 y lo sube a Vocaroo/Archive.org para obtener su enlace directo.
- **Playlist y Expansión de Loop en JavaScript:** En su proyecto de VS Code, el código contiene un arreglo (*Array*) llamado playlistRadio donde el alumno simplemente pega y actualiza los enlaces de los bloques producidos (Hora 1, Hora 2, Hora 3...).
- **Bucle Infinito Acumulativo:** El reproductor web reproduce los enlaces en secuencia automática (Bloque 1 $\rightarrow$ Bloque 2 $\rightarrow$ Bloque 3). Cuando termina el

último bloque disponible, el sistema **reinicia automáticamente la transmisión desde el primer enlace**, creando un ciclo en loop continuo que se va expandiendo en tiempo y contenido conforme el alumno añade más horas a su lista.

3. Estructura del Proyecto en VS Code

La carpeta raíz del proyecto se llamará **"MiRadioEnVivo"** y contendrá la siguiente organización de archivos y recursos:

- **Archivos principales:** index.html, style.css y script.js en la raíz del proyecto.
- **Carpeta de Recursos (assets/):** Subcarpeta que almacena los artes visuales de la estación:
  - **assets/logo.jpg** (Logotipo de Rizz Radio).
  - **assets/portada.jpg** (Carátula de transmisión actual).

4. PROMPT PARA RIZZ RADIO EN GEMINI

"Actúa como un desarrollador Frontend Senior experto en HTML5, CSS3 y JavaScript moderno. Necesito que crees la interfaz web y la lógica de programación para una estación de radio digital juvenil llamada 'RIZZ RADIO'.

ESTRUCTURA DE ARCHIVOS Y RUTAS LOCALES:

La carpeta raíz del proyecto se llama MiRadioEnVivo.
Dentro de la raíz se ubican los archivos: index.html, style.css y script.js.
Los recursos visuales están dentro de la carpeta assets/:
- Logotipo principal: assets/logo.jpg
- Portada de la transmisión: assets/portada.jpg

REQUERIMIENTOS DE DISEÑO Y MAQUETACIÓN (UI/UX):

1. Diseño de Cabina / Reproductor Digital: Interfaz moderna, minimalista y oscura estilo plataforma de streaming actual (tonos grafito profundo y acentos en morado neón vibrante y cian), con contenedores en glassmorphism y un ecualizador gráfico animado con CSS.
2. Encabezado e Indicador en Vivo: Mostrar assets/logo.jpg junto al nombre 'RIZZ RADIO' y una insignia parpadeante 'EN VIVO / ON AIR'.
3. Módulo Principal de Transmisión: Contenedor destacado que muestra la carátula principal (assets/portada.jpg), el nombre del bloque actual de la programación y el tiempo de reproducción.
4. Panel de Control Estilo Streaming (Controles Avanzados):
   - Botón Play / Pause grande y estilizado.
   - Botones de 'Anterior' y 'Siguiente' para saltar entre los bloques horarios de la parrilla.
   - Barra de progreso interactiva (Seekbar) con el tiempo transcurrido y duración total.
   - Control de volumen deslizante.
   - Selector de velocidad de reproducción (1x, 1.25x, 1.5x, 2x) para adaptarse a los hábitos de escucha actuales de los jóvenes.
5. Parrilla de Programación Visual: Un listado en pantalla que muestre los bloques cargados dinámicamente indicando visualmente cuál está al aire.

REQUERIMIENTOS FUNCIONALES (JAVASCRIPT / PLAYLIST Y LOOP GLOBAL):

1. Gestor de Links de Audio (Playlist - ¡MUY CLARO PARA EL ALUMNO!):
   En el archivo script.js, debes crear un arreglo (Array) llamado exactamente `playlistRadio`. Este arreglo debe venir altamente comentado y visible al inicio del archivo con instrucciones sencillas

para que el alumno entienda cómo copiar, pegar, añadir o quitar entre comillas los enlaces directos de sus audios de Vocaroo o Archive.org (Ejemplo:

```
  const playlistRadio = [
    "AQUÍ_PEGAS_EL_LINK_DE_VOCAROO_DE_LA_HORA_1.mp3",
https://voca.ro/1mCED2ZdDDYF
    "AQUÍ_PEGAS_EL_LINK_DE_VOCAROO_DE_LA_HORA_2.mp3"
https://voca.ro/17LkdsecCIC2
    "AQUÍ_PEGAS_EL_LINK_DE_VOCAROO_DE_LA_HORA_3.mp3"
https://voca.ro/17DYjlJOl41f
```

2. Reproducción Secuencial y Loop Infinito Acumulativo:
- Al reproducir, el sistema toma el primer enlace del arreglo.
- Al terminar un bloque (evento ended), avanza automáticamente al siguiente enlace de la lista.
- Al finalizar el último bloque del arreglo, el sistema reinicia automáticamente la reproducción desde el primer enlace (Hora 1), creando un bucle continuo (loop global) de transmisión que crece conforme se añaden más horas de contenido al arreglo.

3. Caja de Comentarios Interactiva: Un formulario para que los oyentes dejen sus saludos y peticiones musicales en tiempo real guardados en localStorage.

ENTREGABLE REQUERIDO:

Entrega el proyecto separado de manera profesional en TRES códigos completos, sin omitir nada, listos para copiar y pegar directamente en VS Code: index.html, styles.css y script.js.

REGLA DE ENTREGA CRÍTICA PARA EL MODELO: NO utilices ninguna vista previa renderizada, ni canvas visual, ni interfaz ejecutada. DEBES entregar los códigos exclusivamente dentro de bloques de código Markdown tradicionales con el lenguaje especificado (html, css, javascript, json). Para que el alumno pueda ver y usar directamente el botón de 'Copiar código'."

NOTA IMPORTANTE: Si tu proyecto abre en tu PC, pero los botones de Play o acceso a links o enlaces como Vocaroo, o plataformas que ya se usan en Internet, NO ABREN, no te preocupes esto es muy normal, ya que, por seguridad, Internet los bloquea hasta que tus proyectos estén dados de alta con un dominio real y autentico en Internet, y esto lo aprenderemos a hacer en otras lecciones más adelante.